

软件质量保证与测试大题模拟练习_副本

所属课程_无 总分_100.0

一、应用题（共 2 题，每小题 20.0 分，共 40.0 分）

1、某烟酒商店主要销售某种品牌的白酒、红酒和啤酒。白酒、红酒和啤酒的单价分别为 500 元、600 元和 20 元，每瓶白酒的利润为 150（50 瓶以上），200（50 瓶以内）元；每瓶红酒的利润为 200（100 瓶以上），300 元（100 瓶以内）；每瓶啤酒的利润为 5 元（500 瓶以上），10 元（500 瓶以内）。商店每月最多只允许销售 100 瓶白酒、200 瓶红酒和 1000 瓶啤酒。根据销售业绩，商店每月计算利润收入。使用一般边界值分析法进行测试用例的设计

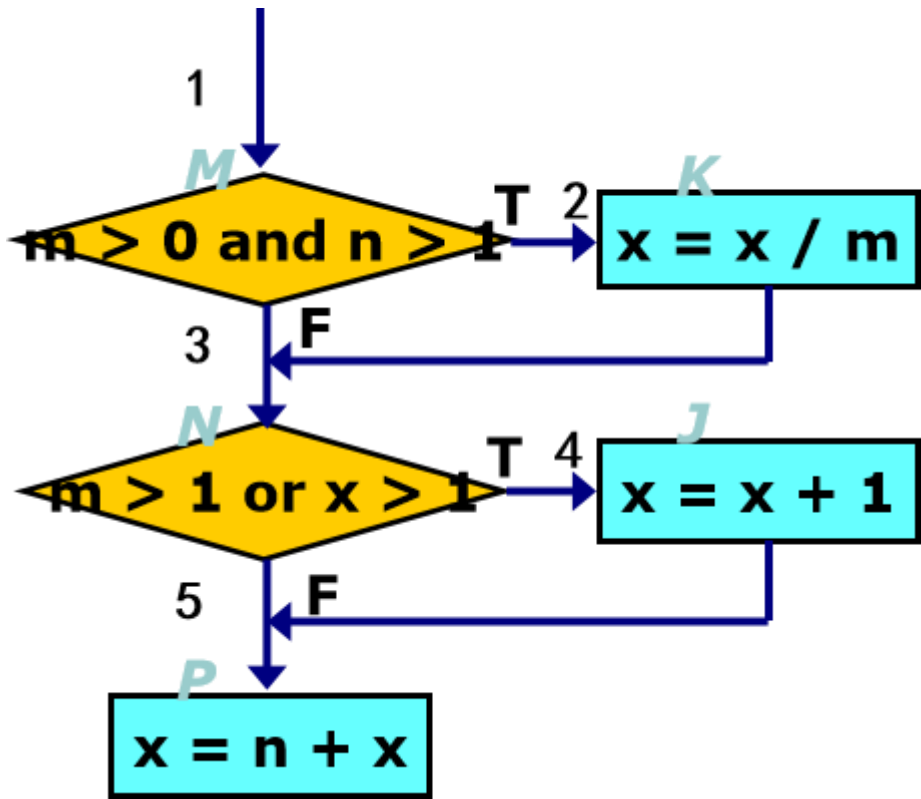
答：

至少 4n+1

ID	输入数据（单位：瓶）			预期输出（单位：元）	备注
	白酒	红酒	啤酒		
01	0	100	500	略	白酒的边界
02	1	100	500	略	白酒的边界
03	99	100	500	略	白酒的边界
04	100	100	500	略	白酒的边界
05	50	0	500	略	红酒的边界
06	50	1	500	略	红酒的边界
07	50	199	500	略	红酒的边界
08	50	200	500	略	红酒的边界
09	50	100	0	略	啤酒的边界
10	50	100	1	略	啤酒的边界
11	50	100	999	略	啤酒的边界
12	50	100	1000	略	啤酒的边界
13	0	0	0	略	整体边界
14	1	0	0	略	整体边界
15	0	1	0	略	整体边界
16	0	0	1	略	整体边界
17	100	200	1000	略	整体边界

18	99	200	1000	略	整体边界
19	100	199	1000	略	整体边界
20	100	200	999	略	整体边界

2、对下列程序进行判定条件覆盖



答：

①判定条件覆盖分析如下 判定 $m > 0$ and $n > 1$ 为真时记为 M,为假时记为-M 判定 $m > 1$ or $x > 1$ 为真时记为 N,为假时记为-N 条件 $m > 0$ 为真时记为 T1,为假时记为 F1 条件 $n > 1$ 为真时记为 T2,为假时记为 F2 条件 $m > 1$ 为真时记为 T3,为假时记为 F3 条件 $x > 1$ 为真时记为 T4,为假时记为 F4

测试用例	条件取值	具体取值条件	判定取值	通过路径
------	------	--------	------	------

输入：m=4，n=2，x=8 输出：m=4，n=2，x=5	T1，T2， T3，T4	m>0,n>1, m>1，x>1	M N	P1（1-2-4）
输入：m=-2，n=0，x=1 输出：m=-2，n=0，x=1	F1，F2， F3，F4	m<=0, n<=1, m<=1, x<=1	-M -N	P4（1-3-5）

二、测试编程题（共 3 题，每小题 20.0 分，共 60.0 分）

1、 现有一个 Factorial 类用于计算阶乘结果。作为测试工程师，你需要编写参数化测试来验证该方法的正确性。被测试类代码如下（请勿修改）：

```
public class Factorial {
    public long fact(long n) {
        long r = 1;
        for (long i = 1; i <= n; i++) {
            r = r * i;
        }
        return r;
    }
}
```

要求：

- (1) 编写一个偶数参数化测试类，包含至少 2 个偶数（5 分）
- (2) 编写一个奇数参数化测试类，包含至少 2 个奇数（5 分）
- (3) 编写一个 测试套件类 FactorialTestSuit，将上述两个测试类包含在内，使得运行该套件可以一次性执行所有测试。（2 分）
- (4) 提交全部 3 个测试类代码以及测试套件类运行结果截图。（3 分）

答：

偶数参数化测试类：

```
@RunWith(Parameterized.class)
```

```
public class FactorialTestDouble {
    private long num;
    private long expectResult;
    public FactorialTestDouble(long num, long expectResult) {
        this.num = num;
        this.expectResult = expectResult;
    }
    @Parameterized.Parameters
    public static Collection<Object[]> data() {
        return Arrays.asList(new Object[][] {
            {2, 2},
            {4, 24}
        });
    }
}
```

```
@Test
public void fact() {
    Factorial factorial=new Factorial();
    assertEquals(expectResult,factorial.fact(num));
}
}
```

奇数参数化测试类：

```
@RunWith(Parameterized.class)
public class FactorialTestSingle {
    private long num;
    private long expectResult;
    public FactorialTestSingle(long num, long expectResult) {
        this.num = num;
        this.expectResult = expectResult;
    }
    @Parameterized.Parameters
    public static Collection<Object[]> data() {
        return Arrays.asList(new Object[][] {
            {3, 6},
            {5, 120}
        });
    }
}
```

```
@Test
public void fact() {
```

```

Factorial factorial=new Factorial();
assertEquals(expectResult,factorial.fact(num));
}
}

```

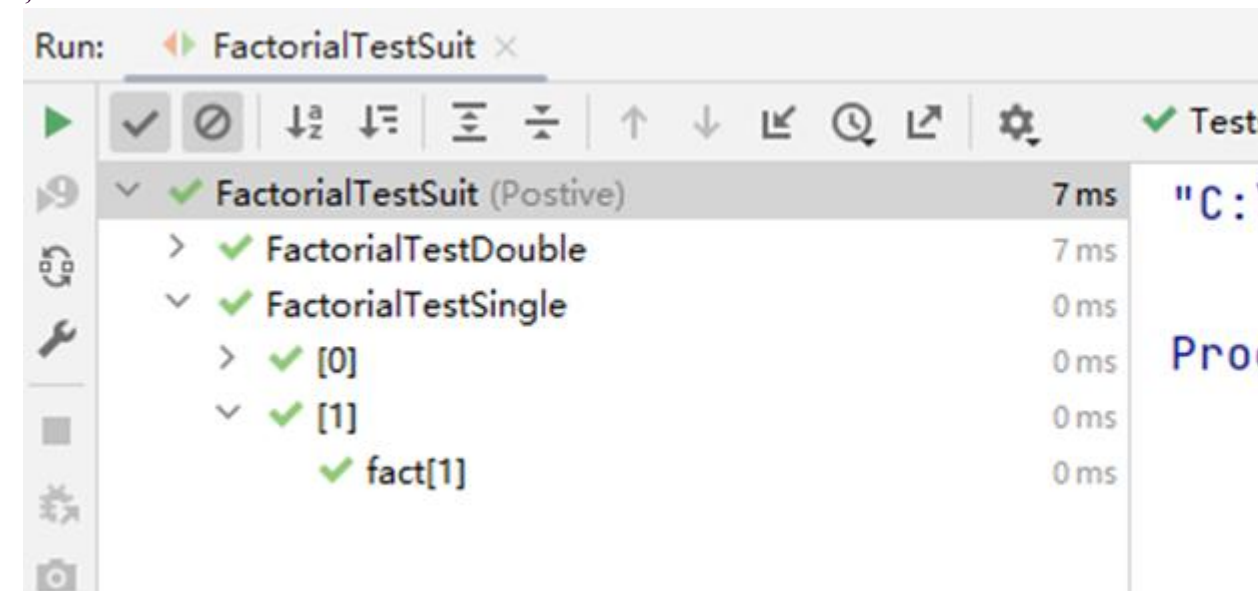
测试套件类:

```

@RunWith(Suite.class)
@Suite.SuiteClasses({
    FactorialTestDouble.class,
    FactorialTestSingle.class
})
public class FactorialTestSuit {

}

```



2、有如下数组处理类请不要修改 ArrayUtils 类的任何代码，只需编写测试类对其进行验证并截图测试运行结果。

```

public class ArrayUtils {

    public int findMax(int[] arr) {

        if (arr == null || arr.length == 0) {

```

```

            throw new IllegalArgumentException("数组不能为 null 或空");
        }

        int max = arr[0];

        for (int num : arr) {

            if (num > max) {

                max = num;

            }

        }

        return max;
    }

    public int findMin(int[] arr) {

        if (arr == null || arr.length == 0) {

            throw new IllegalArgumentException("数组不能为 null 或空");

        }

        int min = arr[0];

        for (int num : arr) {

            if (num < min) {

                min = num;

            }

        }

        return min;
    }

    public int sum(int[] arr) {

```

```

    if (arr == null) {

        return 0;

    }

    int total = 0;

    for (int num : arr) {

        total += num;

    }

    return total;

}

public int average(int[] arr) {

    return sum(arr) / arr.length;

}

}

```

要求:

使用@Before 注解初始化 ArrayUtils 以及用于测试的数组 {67, 434, 98, 2, 4}。

使用@After 注解把测试对象与测试数组置为 null 释放资源。

测试最大值

测试最小值

测试求和

测试求平均

提交整个测试类代码以及测试运行结果截图。

答:

```

public class ArrayUtilsTest {
    private int[] a;
    private ArrayUtils arrayUtils;

```

```

@Before
public void setUp() throws Exception {
    a= new int[] {67, 434, 98, 2, 4};
    arrayUtils=new ArrayUtils();
}

```

```

@After
public void tearDown() throws Exception {
    a=null;
    arrayUtils=null;
}

```

```

@Test
public void findMax() {
    assertEquals(434,arrayUtils.findMax(a));
}

```

```

@Test
public void findMin() {
    assertEquals(2,arrayUtils.findMin(a));
}

```

```

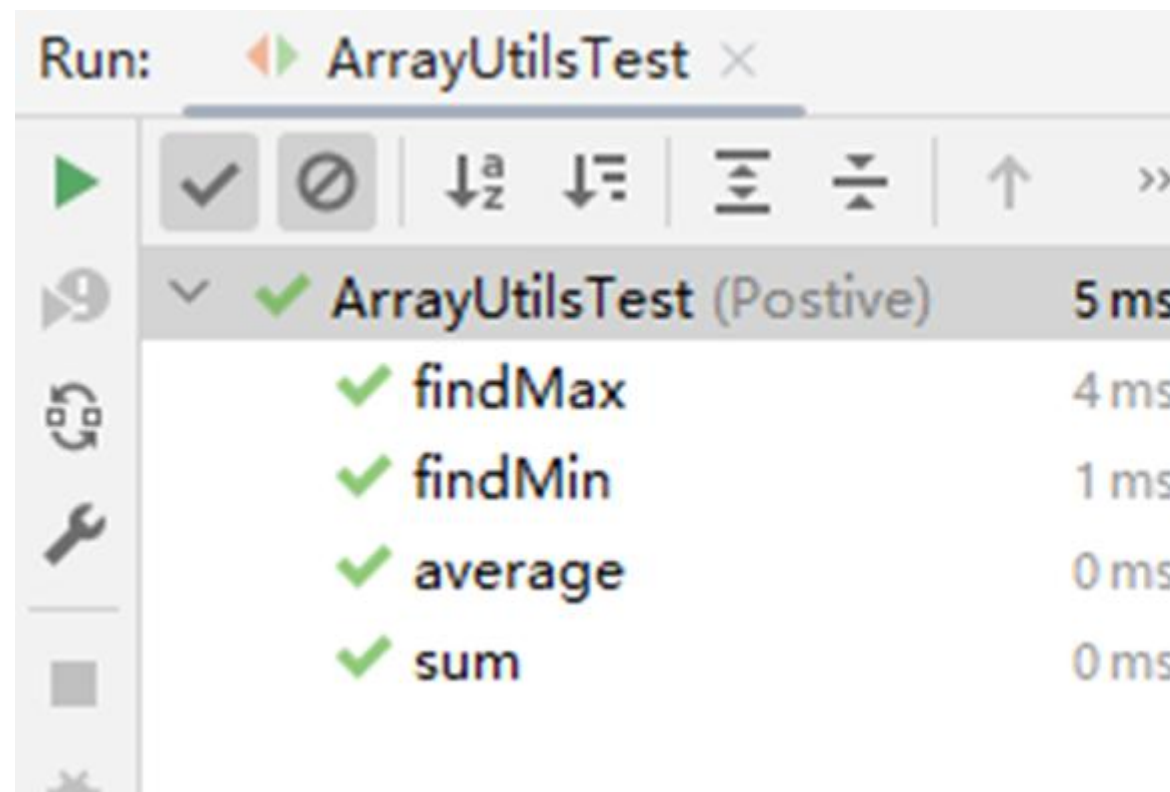
@Test
public void sum() {
    assertEquals(605,arrayUtils.sum(a));
}

```

```

@Test
public void average() {
    assertEquals(121,arrayUtils.average(a));
}
}

```



3、有如下计算器类 Calculator，请不要修改 Calculator 类的任何代码，只需编写测试类对其进行验证并截图测试运行结果。

```
public class Calculator {

    public static void main(String[] args) {

    }

    public int add(int a, int b) {

        return a + b; // 简单的加法运算

    }

    public int minus(int a, int b) {

        return a - b; // 减法运算

    }

    public int multi(int a, int b) {

        return a * b; // 乘法运算

    }

    public int divd(int a, int b) throws Exception {

        return a / b; // 除法运算

    }

}
```

```
}

public int multi(int a, int b) {

    return a * b; // 乘法运算

}

public int divd(int a, int b) throws Exception {

    return a / b; // 除法运算

}

}
```

要求:

- (1) 使用@Before 注解初始化 Calculator。
- (2) 使用@After 注解把测试对象置为 null 释放资源。
- (3) 测试加减乘除操作
- (4) 提交整个测试类代码以及测试运行结果截图。

答:

```
public class CalculatorTest{

    private Calculator calculator;

    @Before

    public void setUp() {

        calculator = new Calculator();

    }

    @After

    public void tearDown() throws Exception {

        calculator = null;

    }

}
```

```
}

@Test
public void add() {
    Assert.assertEquals(3, calculator.add(1, 2));
}

@Test
public void minus() {
    Assert.assertEquals(-1, calculator.minus(1, 2));
}

@Test
public void multi() {
    Assert.assertEquals(2, calculator.multi(1, 2));
}

@Test
public void divd(){
    Assert.assertEquals(4, calculator.divd(4, 1));
}
```

